

Universidad Tecnológica Nacional

Facultad Regional Villa Maria



Trabajo Práctico Integrador 1

Materia: Programación/Matemática

Integrantes: Genaro Dagatti - Benjamin Iovanovich - Lautaro Juarez
Primer Año - Comisión 2

Profesores: Francisco Becerra - Matias Corletti

Tecnicatura Universitaria en Programación 2026

Consigna 1 — Validación de usuarios y análisis de consistencia del sistema

1)

Calcular e interpretar:

$A = \{101, 102, 103, 104, 105, 106\}$

$B = \{104, 105, 106, 107, 108\}$

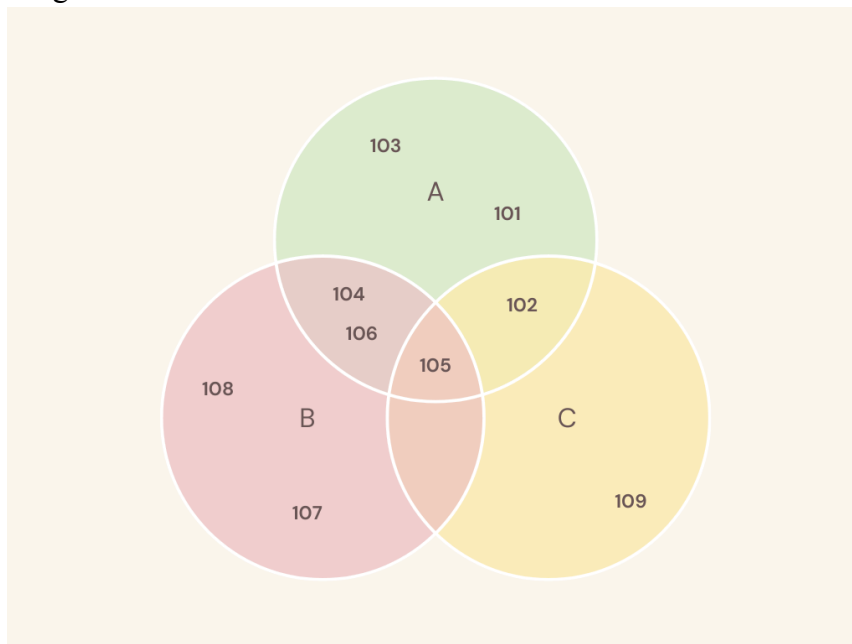
$C = \{102, 105, 109\}$

A: usuarios que acceden mediante API.

B: usuarios que acceden mediante web.

C: usuarios que presentan errores.

Diagrama de Vann:



2)

A) $A \cap B = \{X \mid X \in A \wedge X \in B\}$

$A \cap B = 104, 105, 106$

B) $A \cup B - C = \{X \mid X \in A \cup B \wedge X \notin C\}$
107, 108

$A \cup B - C = 101, 103, 104, 106,$

A) Se lee: x tal que x pertenece a “A” y x pertenece a “B”

B) Se lee: x tal que x pertenece a la unión de “A” y “B” y x no pertenece a “C”

3) Los usuarios {102,105,109} aparecen en “C” pero no en “A” ∪ “B”

4)

p: el usuario pertenece a “A” (usa Api)

q: el usuario pertenece a “B” (usa Web)

r: el usuario pertenece a “C” (genera errores)

$$(p \vee q) \wedge r$$

Significa: Un usuario es crítico si usa al menos una plataforma (API o Web) y también genera errores.

5) Tabla de la verdad:

p	q	r	$p \vee q$	$(p \vee q) \wedge r$
V	V	V	V	V
V	V	F	V	F
V	F	V	V	V
V	F	F	V	F
F	V	V	V	V
F	V	F	V	F
F	F	V	F	F
F	F	F	F	F

6)

```
1 A = {101, 102, 103, 104, 105, 106}
2 B = {104, 105, 106, 107, 108}
3 C = {102, 105, 109}
4
5 todos = A | B | C
6
7 criticos = []
8 no_criticos = []
9
10 for usuario in todos:
11     p = usuario in A
12     q = usuario in B
13     r = usuario in C
14
15     if (p or q) and r:
16         criticos.append(usuario)
17     else:
18         no_criticos.append(usuario)
19
20
21 print("Usuarios criticos: ", criticos)
22 print("Usuarios no criticos: ", no_criticos)
```

```
Usuarios criticos: [102, 105]
Usuarios no criticos: [101, 103, 104, 106, 107, 108, 109]
```

7) PS C:\Users\Usuario\Python> □

8)

- A) Los usuarios críticos (102,105) porque son los que generan errores en el sistema
- B) Significa que el usuario generó errores pero no hay un registro que haya usado el sistema
- C) Deberíamos analizar porque el usuario {109} genera errores sin usar el Api ni la Web y implementar un sistema de validación y monitoreo

Consigna 2 — Análisis y optimización de costos de desarrollo

Función “A”

$$A(x) = 40x + 200$$

- Pendiente: 40
- Ordenada al origen: (0 , 200)

El plan “A” tiene un costo fijo de \$200 y aumenta \$40 por cada hora utilizada.

Función “B”

$$B(x) = 70x + 50$$

M (pendiente) = cuánto sube el costo por cada hora extra

B (ordenada al origen) = cuánto se paga aunque se usen 0 horas (costo fijo)

- Pendiente: 70
- Ordenada al origen: (0 , 50)

El plan “B” tiene un costo fijo de \$50 y aumenta \$70 por cada hora utilizada.

¿Son paralelas?

Dos rectas son paralelas si suben a la misma velocidad, es decir, si tienen la misma pendiente

A sube de a \$40 por hora

B sube de a \$70 por hora

Cálculo punto de intersección entre “A” y “B”:

$$A(x) = B(x)$$

$$40x + 200x = 70x + 50$$

$$200 - 50 = 70x - 40x$$

$$150 = 30x$$

$$x = 5$$

$$A(5) = 40(5) + 200$$

$$A(5) = 200 - 200$$

$$A(5) = 400$$

$$(5, 400)$$

vértice:

El vértice nos dice cuántas horas dan el costo máximo y cuánto es ese costo. La fórmula del vértice es: $x_v = -b / (2a)$ con $a = -2$, $b = 80$, $c = 100$

Paso a paso:

$$(X-V) = -80 / (2 \times -2) = -80 / -4 = \mathbf{20}$$

Ahora calculamos el costo en $x = 20$:

$$C(20) = -2(20)^2 + 80(20) + 100 = -2(400) + 1600 + 100 = -800 + 1600 + 100 = \mathbf{900}$$

Vértice: (20;900) Con 20 horas el costo máximo es **(900)**

Raíces: dónde la función vale cero Las raíces nos dicen en qué cantidad de horas el costo sería \$0 (puntos de corte con el eje X)

usamos la fórmula cuadrática: $(x = (-b \pm \sqrt{(b^2 - 4ac)}) / (2a))$

Calculando el discriminante (Δ):

$$\Delta = 80^2 - 4(-2)(100) = 6400 + 800 = \mathbf{7200}$$

$$\sqrt{7200} = \sqrt{(3600 \times 2)} = 60\sqrt{2} \approx \mathbf{84.85}$$

Dos raíces:

$$x_1 = (-80 + 84.85) / (-4) = 4.85 / -4 \approx -1.21$$

$$x_2 = (-80 - 84.85) / (-4) = -164.85 / -4 \approx 41.21$$

Raíces: $x \approx -1.21$ y $x \approx 41.21$

Como el dominio es $0 \leq x \leq 50$, solo $x \approx 41.21$ tiene sentido práctico: a partir de las 41 horas, el modelo C daría costos negativos (lo que indica que el plan deja de ser aplicable más allá de ese punto).

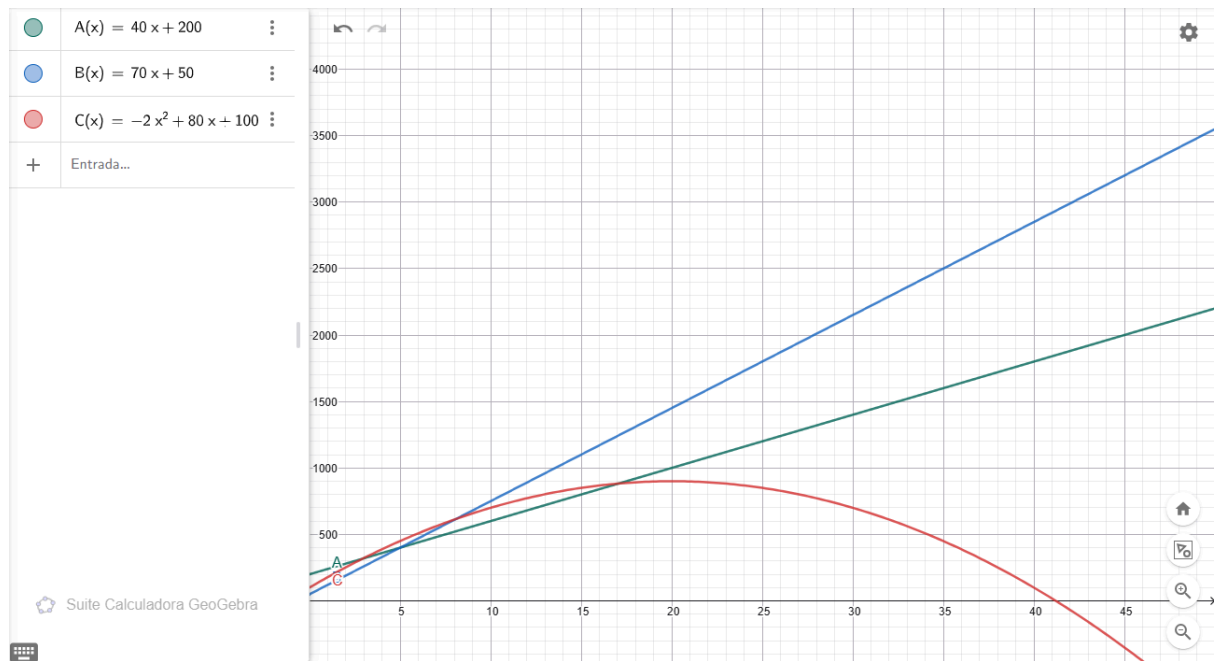
Vértice (máximo)
(20, 900)
Raíz dentro del dominio
$x \approx 41.21$
Raíz fuera del dominio
$x \approx -1.21$

Parte B:

Funciones en python:

```
consignab.py > ...
1  x = [0,5,10,15,20,25,30,40,50]
2
3  A = []
4  B = []
5  C = []
6
7  for i in range(len(x)):
8
9      a = 40*x[i] + 200
10     b = 70*x[i] + 50
11     c = -2*(x[i]**2) + 80*x[i] + 100
12
13     A.append(a)
14     B.append(b)
15     C.append(c)
16
17 for i in range(len(x)):
18     print("x =",x[i])
19     print("A =",A[i])
20     print("B =",B[i])
21     print("C =",C[i])
22     print("-----")
23
24 horas = int(input("Ingrese cantidad de horas: "))
25
26 a = 40*horas + 200
27 b = 70*horas + 50
28 c = -2*(horas**2) + 80*horas + 100
29
30 if a <= b and a <= c:
31     print("Plan más económico: A")
32     print("Costo:",a)
33 elif b <= a and b <= c:
34     print("Plan más económico: B")
35     print("Costo:",b)
36 else:
37     print("Plan más económico: C")
38     print("Costo:",c)
```

Gráfico en GeoGebra:



Parte C:

9) Luego de comparar los costos de los tres planes en el dominio:

$$0 \leq x \leq 50$$

se obtiene:

Plan B conviene para:

$$0 \leq x < 5$$

Plan A conviene para:

$$5 < x < 17.07$$

Plan C conviene para:

$$17.07 < x \leq 50$$

En:

$$x = 5$$

Los planes A y B tienen el mismo costo.

10) La función:

$$C(x) = -2x^2 + 80x + 100$$

Toma valores negativos para:

$$41.21 < x \leq 50$$

Explicación:

Esto es un problema real porque un costo negativo significa que la empresa estaría entregando dinero al cliente por utilizar el servicio. En una situación real eso no tiene sentido económico, por lo tanto el modelo deja de ser válido para esos valores.

Consigna 3 — Análisis de rendimiento en un sistema distribuido**Parte A — Comprensión del modelo:**

1 - La matriz de tiempos es:

$$M = \begin{pmatrix} 120 & 150 & 100 \\ 200 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix}$$

Cada fila representa una función del sistema.

Cada columna representa un servidor.

Cada valor representa el tiempo promedio, en milisegundos, que tarda una función en ejecutarse en un servidor.

2 - Las dimensiones son:

$$M = 3 \times 3$$

$$C = 3 \times 3$$

Sí se puede calcular: $M \times C$

porque el número de columnas de M es igual al número de filas de C .

Parte B — Implementación en Python:

```
Untitled-1.txt •  Untitled-1.py •  total = 0.py
Untitled-1.py > ...
1  |
2  M = [
3  |   [120, 150, 100],
4  |   [200, 180, 220],
5  |   [90, 110, 95]
6  | ]
7
8
9  C = [
10 |   [30, 20, 10],
11 |   [15, 25, 20],
12 |   [40, 10, 30]
13 | ]
14
15
16 print("Promedio por función:")
17
18 for fila in M:
19     promedio = sum(fila) / len(fila)
20     print(promedio)
21
22
23 print("\nPromedio por servidor:")
24
25 for columna in range(3):
26     suma = 0
27
28     for fila in range(3):
29         suma += M[fila][columna]
30
31     promedio = suma / 3
32     print(promedio)
33
34
35 print("\nMatriz transpuesta:")
36
37 for columna in range(3):
```

```
35     print("\nMatriz transpuesta:")
36
37     for columna in range(3):
38         for fila in range(3):
39             print(M[fila][columna], end=" ")
40
41         print()
42
43
44
45     print("\nProducto M * C:")
46
47     T = []
48
49     for i in range(3):
50         fila_resultado = []
51
52         for j in range(3):
53             suma = 0
54
55             for k in range(3):
56                 suma += M[i][k] * C[k][j]
57
58             fila_resultado.append(suma)
59
60         T.append(fila_resultado)
61
62
63
64
65     for fila in T:
66         print(fila)
67
```

- **Promedio de ejecución por función**

Función 1:

$$(120 + 150 + 100) / 3 = 123,33 \text{ ms}$$

Función 2:

$$(200 + 180 + 220) / 3 = 200 \text{ ms}$$

Función 3:

$$(90 + 110 + 95) / 3 = 98,33 \text{ ms}$$

Promedio de ejecución por servidor

Servidor 1:

$$(120 + 200 + 90) / 3 = 136,67 \text{ ms}$$

Servidor 2:

$$(150 + 180 + 110) / 3 = 146,67 \text{ ms}$$

Servidor 3:

$$(100 + 220 + 95) / 3 = 138,33 \text{ ms}$$

- La matriz transpuesta de M reorganiza los datos cambiando filas por columnas. Esto permite ver los tiempos de ejecución desde el punto de vista de los servidores en lugar de las funciones.

Parte C — Análisis mediante producto matricial: Calcular el producto $T = M * C$

Fila 1:

$$T11 = 120 \times 30 + 150 \times 15 + 100 \times 40 = 3600 + 2250 + 4000 = \mathbf{9850}$$

$$T12 = 120 \times 20 + 150 \times 25 + 100 \times 10 = 2400 + 3750 + 1000 = \mathbf{7150}$$

$$T13 = 120 \times 10 + 150 \times 20 + 100 \times 30 = 1200 + 3000 + 3000 = \mathbf{7200}$$

Fila 2:

$$T21 = 200 \times 30 + 180 \times 15 + 220 \times 40 = 6000 + 2700 + 8800 = \mathbf{17500}$$

$$T22 = 200 \times 20 + 180 \times 25 + 220 \times 10 = 4000 + 4500 + 2200 = \mathbf{10700}$$

$$T23 = 200 \times 10 + 180 \times 20 + 220 \times 30 = 2000 + 3600 + 6600 = \mathbf{12200}$$

Fila 3:

$$T31 = 90 \times 30 + 110 \times 15 + 95 \times 40 = 2700 + 1650 + 3800 = \mathbf{8150}$$

$$T32 = 90 \times 20 + 110 \times 25 + 95 \times 10 = 1800 + 2750 + 950 = \mathbf{5500}$$

$$T33 = 90 \times 10 + 110 \times 20 + 95 \times 30 = 900 + 2200 + 2850 = \mathbf{5950}$$

El resultado es:

[9850 7150 7200]

[17500 10700 12200]

[8150 5500 5950]

T: resume el efecto combinado de aplicar la matriz C sobre cada fila de M

Parte D — Interpretación crítica:

6 - La matriz obtenida del producto $M * C$ es:

T =

| 9850 7150 7200 |

| 17500 10700 12200 |

| 8150 5500 5950 |

Cada valor de la matriz T representa una combinación entre los tiempos promedio de ejecución y las cantidades de ejecuciones registradas.

¿Tiene sentido práctico?

El producto matricial puede servir para realizar análisis matemáticos, pero en este contexto no representa exactamente el tiempo total consumido por cada función o servidor.

7 - La alternativa más adecuada:

Una alternativa más correcta sería multiplicar cada tiempo por la cantidad de ejecuciones correspondiente en la misma posición de la matriz.

Ejemplo:

$$120 \times 30 = 3600$$

$$150 \times 20 = 3000$$

$$100 \times 10 = 1000$$

Esto permitiría obtener el tiempo total consumido por cada función en cada servidor, dando un resultado más útil para analizar el rendimiento.

Parte E — Propiedades de la matriz:

8 -

¿La matriz M es simétrica? No, la matriz M no es simétrica.

Una matriz es simétrica cuando:

$$M = M^T$$

En este caso:

$$M[1][2] = 150$$

$$M[2][1] = 200$$

Como los valores no son iguales, la matriz no es simétrica.

9 - Determinante e inversa

Determinante de M:

$$\det(M) = -152000$$

Como el determinante es distinto de cero, la matriz es invertible.

Por lo tanto, sí es posible calcular la matriz inversa.

10 - Interpretación

Si la matriz no fuera invertible, significa que parte de la información está repetida o relacionada entre sí.

En términos del sistema, esto podría indicar que algunos datos no aportan información nueva o que existe dependencia entre funciones o servidores.

Parte F — Análisis aplicado:

11 - Resultados:

La función con mayor costo computacional promedio es la función 2, con un promedio de 200 ms.

El servidor más eficiente es el servidor 1, ya que posee el menor tiempo promedio de ejecución: 136,67 ms.

12 - Recomendación técnica:

Se recomienda optimizar la función 2, ya que es la que presenta mayor tiempo de ejecución y podría generar cuellos de botella en el sistema.

También sería conveniente distribuir una mayor carga de trabajo al servidor 1, porque es el más eficiente según los resultados obtenidos.

